DL_T03_3A: Anomaly Detection in Bearing Vibration Using Autoencoders

Complex Engineering Problem Task 3, variant 3A (NASA IMS bearing vibration), Team 3. CM701PC / CM751PC Deep Learning, MGIT, B.Tech CSE (AI&ML) VII sem.

Roll number	Role (department allotment)	Main contribution
23261A6660	Data lead	Data pipeline, features, simulator, data card, report sections 2–3
23261A6650	Model lead	Autoencoder models, training, README, section 4
23261A6630	Evaluation and report lead	Metrics, plots, target check, sections 5–7

Roles follow the allotment order; the team must confirm the actual split before submission.

Data status (read first). The NASA IMS archive could not be downloaded in the environment where this project was built. All results below come from a **simulated** IMS style dataset (same file shape, 20 kHz, 10 minute spacing) and describe the **pipeline**, not real bearings. The real-data loader exists and is tested on files in the IMS format, but has not been run on the real archive.

1. Problem understanding

Breakdowns are rare, so a warning system must learn *normal* behaviour only and flag departures. The targets pull against each other: recall ≥ 0.85 on fault readings pushes the alarm threshold down; fewer than 2 false alarms per 24 h pushes it up; detection must come within 3 readings (the task's rule for data sampled slower than once a minute, here 10 minutes); no labelled fault may be used for training. A missed fault costs Rs 4 lakh and a false alarm Rs 2,000, a 200:1 ratio. Constraints I identified: only an unsupervised model is allowed; fault onset is not labelled in the real data, so any labels are an assumption; with one fault per run, test statistics rest on few events (3 test faults here).

2. Data preparation and exploratory analysis

Simulator (`src/imsae/simulate.py`): 8 run-to-failure runs × 4 bearings (700 snapshots each, 20480 samples, 4.9 days). Healthy bearings: noise, shaft harmonics at 33.3 Hz, a 6% daily amplitude cycle and benign transient bursts (≈1.7 per day) that imitate faults. One bearing per run develops an outer-race defect: impulses at 236.4 Hz ringing a 3.5 kHz resonance, amplitude 0.25σ at onset growing 40×. Split by **run**: 3 train, 2 validation, 3 test (seed-fixed). **Features**: per bearing 11 numbers per snapshot (log RMS, kurtosis, crest factor, 8 band log-powers). Each series is standardised with its own first 40% (assumed healthy; no labels). Training uses only that prefix (last 20% of it for early stopping). Validation runs choose thresholds; test runs give every reported number. Class balance: about 5–10% of readings are faulty. Fig. 2 shows the key EDA finding: the error of a healthy-trained model stays flat until well after the labelled onset.

3. Models and experiments

All are Keras autoencoders trained on healthy readings (Adam 1e-3, batch 32, ≤100 epochs, early stopping, 3 seeds 0–2). Score = reconstruction MSE of the newest reading. **dense_wide**: 11→64→11→64→11 (bottleneck = input). **dense_narrow**: bottleneck 8. **dense_dae_sparse**: bottleneck 16, input noise 0.2, L1 activity penalty 1e-3 (noise robustness and sparsity). **lstm_dae** (my "improved" model): LSTM encoder/decoder over a causal window of T=6 readings, bottleneck 8, input noise 0.1. The alarm fires when the score exceeds a threshold for m consecutive readings (m ∈ (1, 2, 3) and the threshold are

chosen on validation data only). Two thresholds are compared: best reading-level F1, and minimum total cost (Rs 4 lakh per *missed* fault + Rs 2,000 per false alarm episode).

Cost model change (disclosed). My first version counted an alarm later than 3 readings as a miss. Every model then scored "missed", so "never alarm" became cheapest, which is meaningless. I changed it after seeing those first results: a fault is *missed* only if never alarmed before the run ends; timeliness is reported separately as "in time" (alarm within 3 readings). All tables use the corrected model.

Model	ROC-AUC	Best-F1 threshold (validation)					Minimum-cost threshold (validation)		
		recall	FA/day	F1	delay	in time	recall	FA/day	delay
dense_wide	0.825 ± 0.020	0.60 ± 0.01	1.01 ± 0.18	0.69 ± 0.01	49	0/9	0.18 ± 0.03	0.00	117
dense_narrow	0.853 ± 0.004	0.64 ± 0.02	0.63 ± 0.23	0.73 ± 0.01	45	0/9	0.24 ± 0.02	0.00	108
dense_dae_sparse	0.866 ± 0.007	0.65 ± 0.02	0.90 ± 0.24	0.73 ± 0.02	45	0/9	0.23 ± 0.04	0.00	109
lstm_dae	0.831 ± 0.002	0.57 ± 0.00	0.70 ± 0.09	0.69 ± 0.01	58	0/9	0.16 ± 0.03	0.00	120

Table 1. Test results, mean ± std over 3 seeds (3 test fault events). FA/day = false alarm episodes per 24 h on healthy data; delay = mean readings from onset to first alarm (1 reading = 10 min); in time = faults alarmed within 3 readings, summed over seeds (3 events × 3 seeds).

4. Results against every target

Target	Result (best model, test)	Met?
Recall on fault readings ≥ 0.85	0.65 at best-F1 point (dense_dae_sparse); 0.83–0.85 only at ≈ 33 FA/day	**No**
False alarms < 2 per 24 h	0.6–1.0 at best-F1; 0.0 at min-cost point; ≈ 33 when recall 0.85 is forced	Yes, but not together with recall
Detection within 3 readings	0 of 3 faults at best-F1 and min-cost points (mean delay 45 and 108 readings); 2–3 of 3 when recall 0.85 is forced	**No** at the usable points
No labelled faults in training	Training uses only the first 40% of each series; labels only for thresholds and testing	Yes

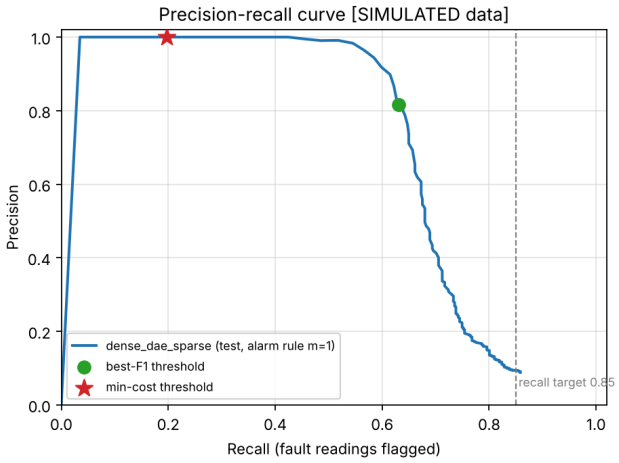


Fig. 1. Precision-recall curve (test, dense_dae_sparse, seed 0) with both operating points. Even the best curve never reaches recall 0.85 above precision ≈ 0.1.

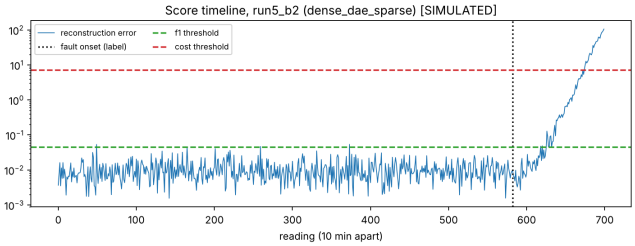


Fig. 2. Score timeline of a faulty bearing: the score leaves the healthy band only ≈ 40 readings after the labelled onset.

	Best F1 (val.)	Min cost (val.)	Recall 0.85 (val.)*
threshold (error)	0.046	7.29	≈0.01
recall / precision	0.63 / 0.82	0.20 / 1.00	0.83–0.85 / 0.11

false alarms per day	1.01	0.00	30.6–33.5
missed faults (never alarmed)	0	0	0
alarmed within 3 readings	0/3	0/3	2–3 of 3
inspection cost (Rs)	112,000	0	≈34,00,000–37,00,000 total

*3 seeds, dense_dae_sparse; the other models reached recall 0.85 on validation in 1 of 3 (narrow) or 0 of 3 (LSTM) seeds.

Table 2. Cost analysis, dense_dae_sparse seed 0 on the 3 test faults (about 55 healthy days).

5. Answers to the report questions

Q1. Why can an autoencoder with a bottleneck as wide as its input fail? With no compression a network with a hidden layer wider than the input can represent the identity map: capacity exceeds what the healthy data needs, so the training error becomes tiny (0.0002 versus 0.0098 at width 8 and 0.096 at width 2) and it also reconstructs mildly abnormal inputs well; this is the overfitting/low-bias end of the capacity curve, and the model has learned nothing about what normal *looks like*. A narrow bottleneck forces the network to keep only the structure shared by healthy data, so deviations from it cost error (the underfitting end appears at width 2, where even healthy error rises to 0.105). Evidence from my runs: AUC is 0.825 for the input-width bottleneck versus 0.876 at width 4 and 0.853 at 8 (Table 3). The gap is modest (about 1–2 seed standard deviations of the wide model, 0.02) and AUC is not monotone, so this supports but does not prove the argument.

bottleneck	AUC	train MSE	healthy MSE
2	0.851	0.0963	0.1054
4	0.876	0.0476	0.0515
8	0.853	0.0098	0.0115
11 (= input)	0.825	0.0002	0.0004
16	0.822	0.0003	0.0005
32	0.830	0.0005	0.0006

Table 3. Dense AE bottleneck sweep (3 seeds, test).

Q2. Did the improved model beat the dense baseline? No. lstm_dae reaches AUC 0.831 ± 0.002 versus 0.853 ± 0.004 for dense_narrow, and the T ablation (T = 1, 3, 6, 12 → AUC 0.853, 0.865, 0.831, 0.829) shows no gain from longer context. Reasons I can support: each reading's 11 features already summarise a full second of vibration, and in this simulator the fault is stationary within a reading and grows smoothly, so what a recurrent model adds (temporal dependence between readings) carries little extra signal; the LSTM has more parameters yet the same 700-reading healthy training set, and it never triggered early stopping before the 100-epoch cap, so it may be under-trained, which I did not test. The best AUC came from the denoising, sparse dense model (0.866 ± 0.007), a small gain (≈ 2 standard deviations) from noise robustness and sparsity. On real IMS data temporal context may matter more; I cannot say.

Q3. Which threshold, given Rs 4 lakh per miss and Rs 2,000 per false alarm? Break-even is 200 false alarms per missed fault, so one expects the cost-optimal threshold to sit *below* the best-F1 one. Here it sits far *above* it (7.29 versus 0.046), because the simulated defect eventually grows so large that even a high threshold catches every fault before the run ends with zero false alarms (Rs 0 on the test runs versus Rs 112,000 for best-F1). The cost model, however, only counts breakdowns and false alarms; it ignores how *late* the warning comes. The min-cost threshold alarms on average 108 readings (≈ 18 h) after onset, the best-F1 one 45 readings (≈ 7.5 h). Because maintenance needs lead time I **recommend the best-F1 threshold** (error ≈ 0.046, m = 1): about one

false alarm a day (inside the limit) and warning ≈ 10 h earlier, for roughly Rs 2,000 a day. Forcing recall to 0.85 needs ≈ 33 false alarms a day (Rs 66,000 a day), far outside the limit; the task's recall and false-alarm targets cannot both be met on this data.

6. Limitations and next steps

(i) **Simulated data**: absolute numbers are properties of my simulator (onset amplitude 0.25σ, 40× growth). A sensitivity run over the onset amplitude (AUC, Table 4) shows the picture depends strongly on it and that the model ranking is stable but small. Real IMS has no onset labels, load changes and sensor drift. (ii) Only 3 test faults and 3 seeds: differences of 0.01–0.03 AUC are within noise. (iii) The "in time" target is unattainable when the labelled onset is below the noise floor; a defensible onset label (for example from RMS or kurtosis trends) is needed for real data. (iv) The cost model changed after first results (section 3). (v) With more time: run on real IMS with literature onset labels, per-bearing thresholds, trend (CUSUM) alarms on the score, and train the LSTM longer.

a0 (σ)	dense_narrow	dense_wide	lstm_dae
0.25	0.855	0.828	0.832
0.5	0.948	0.926	0.931
1.0	0.992	0.990	0.990

Table 4. Test ROC-AUC versus initial defect amplitude a0 (2 seeds): the easier the fault, the higher every model's AUC.

7. References and AI-use declaration

[1] J. Lee, H. Qiu, G. Yu, J. Lin, "Rexnord Technical Services: Bearing Data Set," IMS, Univ. of Cincinnati, NASA Prognostics Data Repository, 2007. [2] H. Qiu, J. Lee, J. Lin, G. Yu, "Wavelet filter-based weak signature detection method and its application on rolling element bearing prognostics," *J. Sound Vib.*, vol. 289, pp. 1066–1090, 2006. [3] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016, ch. 5, 14. [4] D. P. Kingma, J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015. [5] F. Chollet et al., "Keras," 2015.

Appendix: AI use. The code, experiment design and first draft of this report were produced with Claude Code (Anthropic AI coding assistant) in a session with the data lead. All numbers come from the code in the repository and can be regenerated with `python scripts/run_experiments.py`, `scripts/sensitivity_a0.py` and `scripts/recall_target_point.py`. The assistant could not access the real dataset, so it built the simulator; it also proposed the cost-model correction in section 3. Team members must read and be able to explain every line before the viva.